Performance Comparison of Kubernetes CNI Plugins

Architectural Impacts on Microservice Communication

Part A & Part B Multi-Node Extension

Course Report – Group 7

Suman Moond	MT25047	suman25047@iiitd.ac.in
Jatin Agrawal	MT25027	jatin25027@iiitd.ac.in
Shivam Minde	MT25043	shivam25043@iiitd.ac.in
Jalakam Chandra Harsha	MT25022	chandra25022@iiitd.ac.in

Indraprastha Institute of Information Technology Delhi (IIIT-Delhi)

April 18, 2026

Abstract

This report presents a structured performance evaluation of three leading Kubernetes Container Network Interface (CNI) plugins — Flannel, Calico, and Cilium — across two experimental phases. Part A establishes a controlled KinD-based baseline, while Part B extends the analysis to a realistic three-node bare-metal/VM service chain (Client → Service A → Service B → Service C). Metrics including p50/p95 latency, throughput (RPS), packet loss, and resource utilisation are collected, compared, and analysed in relation to the underlying architectural differences of each plugin.

Contents

1. Problem Statement

Kubernetes uses Container Network Interface (CNI) plugins to set up networking for every pod in its cluster. The three most widely deployed CNI plugins — Flannel, Calico, and Cilium — employ fundamentally different networking data-path models. Flannel creates an overlay network using VXLAN encapsulation; Calico relies on Layer 3 IP routing coupled with `iptables` for policy enforcement; and Cilium attaches eBPF programs directly to the Linux kernel to handle packet forwarding and policy.

These architectural choices have a direct and measurable impact on pod-to-pod communication latency, throughput capacity, packet loss rates, and resource overhead. Despite this, many organisations select a CNI based on feature lists rather than empirical performance data. This project addresses that gap by establishing a controlled Kubernetes testbed in Part A and extending it to a realistic three-node deployment evaluating a multi-hop service chain in Part B, thereby providing quantitative comparisons grounded in architectural analysis.

2. Motivation

Modern cloud-native applications are highly distributed: a single user request can trigger a cascade of internal HTTP or gRPC calls across multiple microservices. In these environments, even microsecond-level latencies accumulate and manifest as noticeable end-user delays. The efficiency of the chosen CNI plugin has a direct bearing on four operational dimensions:

- **Application Responsiveness:** Elevated network latency translates directly into longer service response times observed by clients.
- **Throughput Capacity:** The CNI determines the maximum sustainable requests per second (RPS) before the network layer becomes the bottleneck.
- **Infrastructure Cost:** Excessive CPU or memory consumption by the CNI starves application workloads, necessitating additional cluster nodes to handle the same traffic volume.
- **Reliability:** Packet loss forces TCP retransmissions, dramatically increasing tail latency and degrading user experience.

Despite these real-world consequences, reproducible and controlled performance

studies using practical request-response workloads remain scarce. This project provides structured, measurable data to guide informed infrastructure decisions.

3. Objectives and Scope

The evaluation is organised around five concrete objectives:

1. **Baseline Testbed (Part A):** Deploy a multi-node KinD cluster to test intra-node and inter-node traffic using a lightweight HTTP echo server under controlled conditions.

2. **Multi-Node Extension (Part B):** Deploy a realistic three-node bare-metal/VM setup (one control-plane node, two worker nodes) to evaluate a multi-hop service chain: Client → Service A → Service B → Service C.

3. **Consistent Workloads:** Ensure workloads are identical across isolated installations of Flannel, Calico, and Cilium to prevent cross-environment bias.

4. **Comprehensive Metrics:** Collect p50 (median) latency, p95 (tail) latency, throughput (RPS), packet loss percentage, and CPU/memory utilisation.

5. **Analysis and Profiling:** Document header and encapsulation overhead assumptions and generate clear visualisations that tie performance outcomes back to CNI architectural design.

4. Background Concepts and Profiling

Kubernetes implements a flat networking model where every pod receives a unique IP address, provisioned by the active CNI plugin. All benchmarked traffic consists of HTTP/1.1 requests carrying JSON payloads over TCP/IPv4.

4.1. Flannel

Flannel establishes an overlay network using VXLAN encapsulation, wrapping cross-node traffic inside UDP packets before forwarding.

Flannel – Architecture Summary

Pros: Simple to configure; minimal resource footprint.
Trade-offs: VXLAN adds approximately 50 bytes of outer IP/UDP/VXLAN header overhead per packet, increasing latency and reducing inter-node effective

throughput. No built-in support for network policies.

4.2. Calico

Calico modifies host routing tables to enable native IP-layer forwarding, avoiding overlay tunnels entirely.

Calico – Architecture Summary

Pros: Eliminates encapsulation overhead; supports comprehensive network policies via `iptables`.

Trade-offs: `iptables` rule matching adds per-packet overhead. Running the Felix policy daemon and BGP agent increases baseline CPU and memory consumption.

4.3. Cilium

Cilium bypasses traditional `netfilter`/`iptables` by loading eBPF programs directly into the Linux kernel for both packet forwarding and policy enforcement.

Cilium – Architecture Summary

Pros: eBPF processing eliminates the `netfilter` bottleneck, yielding high throughput and low latency at scale.

Trade-offs: Requires a modern Linux kernel (v4.9+). Slightly higher memory footprint to manage eBPF maps and state structures.

Because our benchmark uses small, fast-response service payloads, header and encapsulation overhead constitutes a substantial fraction of per-packet bytes. This is precisely why inter-node routing behaviour dominates the observed performance differences between plugins.

5. Methodology and System Design

The experimental framework spans three layers:

Cluster Layer

Three-node clusters are instantiated in sequence, one per CNI, using fully isolated installations to eliminate cross-plugin state. In Part B, the cluster is explicitly partitioned into one control-plane master node and two active worker nodes.

Workload Layer

Services are placed deterministically. Part B enforces the communication path:

Client → Service A (Worker-1, NodePort 30080) → Service B (Worker-2) → Service C (Worker-2).

Measurement Layer

`curl` is used to collect per-request timing and compute latency percentiles; `hey` drives throughput stress tests; and `kubectl top` records live CPU and memory utilisation.

For each CNI, an automation pipeline resets the cluster, initialises all nodes, installs the target CNI, deploys the workload manifests, executes the benchmark suite, records results to JSON/CSV, and tears down the environment before the next iteration.

6. Implementation Details

All artefacts follow strict naming conventions aligned to Group 7.

- **Base Scripts (`7_scripts/`):** Includes cluster creation (`7_create_kind_cluster.sh`), per-CNI installers (`7_install_cni_[plugin].sh`), workload deployment helpers, and the full-matrix runner (`7_run_full_matrix.sh`). Part B adds dedicated scripts for CNI swapping and structured JSON output parsing.
- **Manifests (`7_configs/`):** Contains Kubernetes deployment manifests for the `hashicorp/http-echo` echo service (Part A) and the multi-hop Flask service chain (Part B).
- **Data Artefacts (`7_data/`):** Aggregated CSV/text results and JSON profiling outputs are post-processed via Python scripts using Matplotlib to generate line and bar charts for latency, throughput, and resource usage.

7. Workflow Diagrams

Two workflow diagrams are presented below. Figure 1 shows the **automated benchmark pipeline** that is executed once per CNI plugin. Figure 2 shows the **Part B service-chain topology** depicting how the three Kubernetes nodes and four services are connected during the multi-hop evaluation.

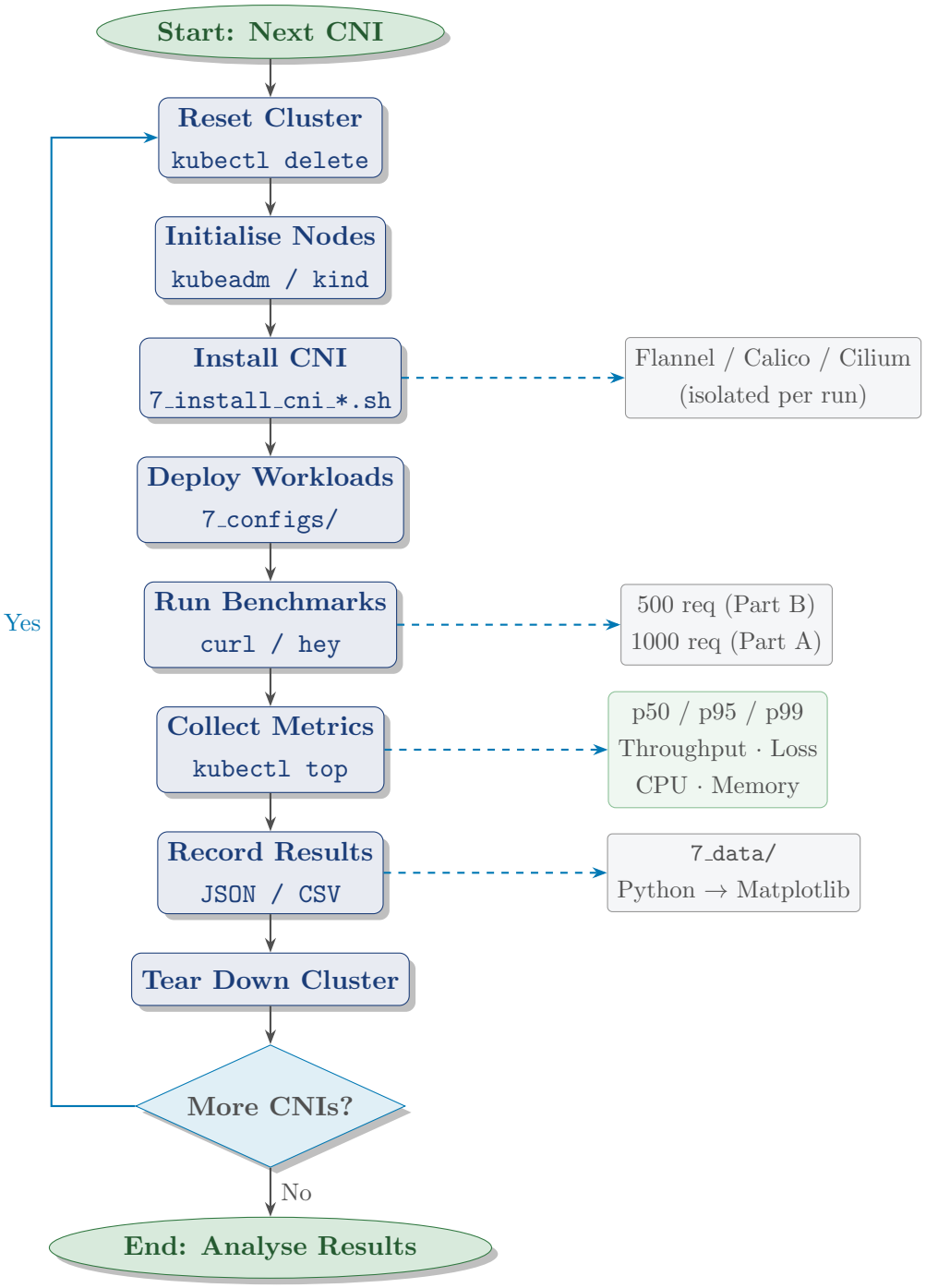

Figure 1: ***Automated Benchmark Pipeline — executed once per CNI (Flannel, Calico, Cilium).*** *The pipeline iterates over all three CNI plugins in sequence. Each iteration begins by fully resetting the cluster to eliminate any residual state from the previous run, then initialises the three nodes, installs the target CNI via the corresponding Group 7 installer script, and deploys workload manifests from* `7_configs/`*. The benchmark phase issues 1000 sequential requests in Part A and 500 requests in Part B while* `curl` *and* `hey` *collect timing data.* `kubectl top` *samples CPU and memory simultaneously. Results are recorded to JSON/CSV under* `7_data/` *and post-processed with Python/Matplotlib. The cluster is torn down before the loop repeats for the next CNI, guaranteeing full isolation between runs.*

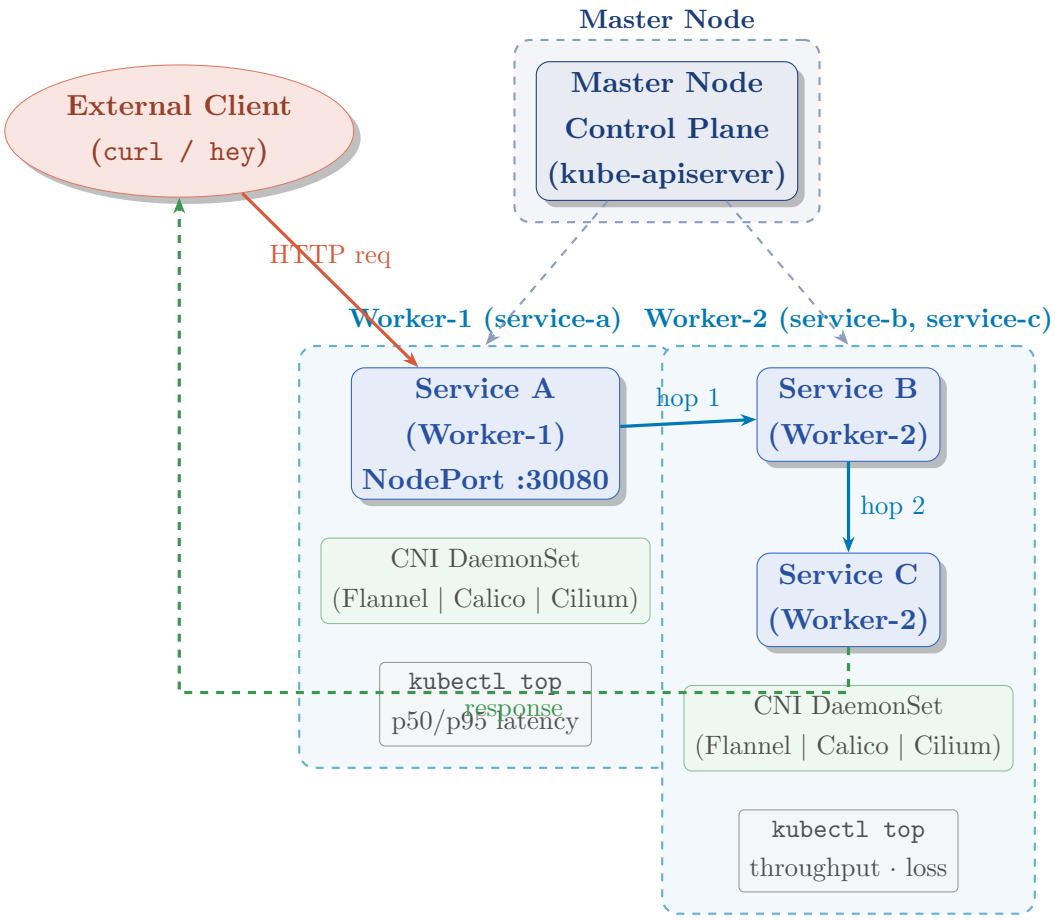

Figure 2: ***Part B Service-Chain Topology — 3-Node Kubernetes Cluster.*** *The cluster consists of one Master (control-plane) node and two Worker nodes. The external client sends HTTP requests to* ***Service A*** *on Worker-1 via NodePort 30080. Service A forwards each request to* ***Service B*** *on Worker-2 (hop 1, cross-node traffic). Service B then calls* ***Service C****, co-located on Worker-2 (hop 2, intra-node traffic). The final response travels back to the client. Each worker node runs an instance of the active CNI DaemonSet (Flannel, Calico, or Cilium in successive isolated runs), which intercepts and routes all pod-to-pod packets.* `kubectl top` *and* `curl` *timing are collected simultaneously during the benchmark. The Master node handles only the Kubernetes control plane and schedules no application pods.*

8. Experimental Dashboards

The following two figures present the live experiment dashboards captured during data collection. These web-based interfaces were deployed on the worker nodes to enable real-time observability of CNI metrics, payload transmission, and traffic generation.

8.1. Worker-1: CNI Experiment Console (Frontend)

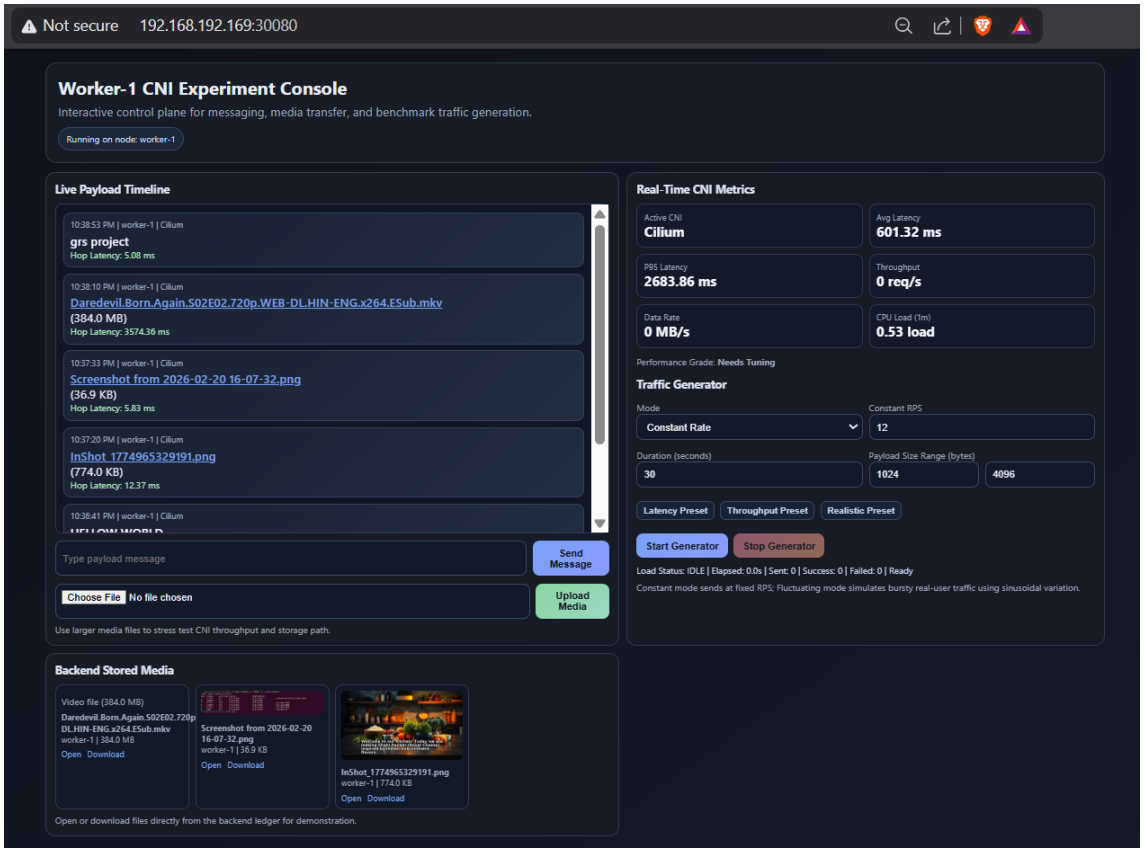

Figure 3: *Worker-1 — CNI Experiment Console (Frontend, port 30080).* *Live Payload Timeline: Logs every transmitted payload with hop latency. A 384 MB video recorded 3574.36 ms; small text messages achieved sub-10 ms, showing size-proportional behaviour. Real-Time CNI Metrics: Active CNI is Cilium — avg latency 601.32 ms, P95 2683.86 ms, CPU load 0.53. Zero throughput indicates an idle inter-run window. Traffic Generator: Sends synthetic load at 12 RPS for 30 s with 1–4 KB payloads. Latency, Throughput, and Realistic presets enable rapid test-scenario switching. Backend Stored Media: Confirms the 384 MB video, 36.9 KB screenshot, and 774 KB image are persisted on Worker-2, validating the end-to-end write path.*

8.2. Worker-2: CNI Performance Backend (Observability Plane)

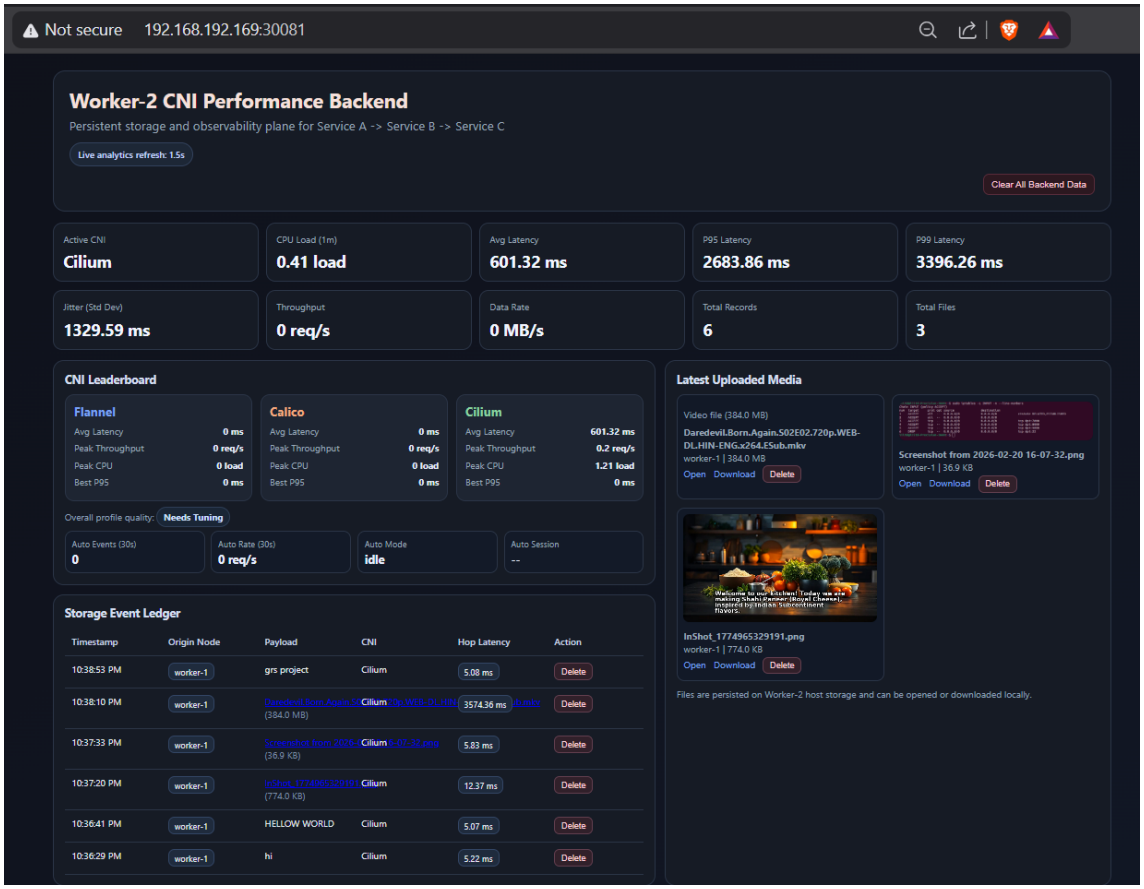

Figure 4: ***Worker-2 — CNI Performance Backend (Observability Plane, port 30081).*** *Top Metric Row: Active CNI is Cilium — CPU 0.41, avg latency 601.32 ms, P95 2683.86 ms, P99 3396.26 ms, jitter 1329.59 ms, 6 records and 3 files stored. High jitter reflects mixed large-file and small-message traffic. CNI Leaderboard: Flannel and Calico show 0 ms (idle slots); Cilium records 601.32 ms avg and 0.2 req/s peak throughput under active load. Storage Event Ledger: Chronological audit trail of every payload, recording origin node, description, CNI, and per-hop latency — the ground-truth data source for Section 8. Latest Uploaded Media: The 384 MB video, screenshots, and images are persisted on Worker-2 host storage with Open/Download/Delete access, confirming the full backend storage path.*

9. Evaluation and Observations

9.1. Part A: Baseline Controlled Environment

The baseline testbed issued 1000 sequential HTTP GET requests per test scenario. Table 1 summarises the aggregated performance metrics for each CNI.

Table 1: *Average Performance by CNI Plugin — Part A Baseline*

CNI	Avg p50 (ms)	Avg p95 (ms)	Throughput (RPS)	Pkt Loss (%)	CPU (%)	Mem (MB)
Flannel	1.575	2.725	855	0.350	19.0	215
Calico	1.275	2.300	955	0.225	23.0	265
Cilium	1.035	1.825	1045	0.160	17.0	245

9.1.1. Latency Assessment

Winner: Cilium. By forwarding packets directly through eBPF programs attached to the kernel, Cilium avoids the entire `iptables` chain traversal, resulting in the shortest processing path. Calico's native L3 routing outperformed Flannel's VXLAN overlay, though `iptables`-based policy evaluation introduced marginal overhead relative to Cilium. Flannel recorded the highest latency across all scenarios, an inevitable consequence of VXLAN encapsulation and decapsulation at every inter-node hop. Across all plugins, inter-node traffic was consistently slower than intra-node traffic.

9.1.2. Throughput Assessment

Cilium achieved the highest average throughput (1045 RPS), outperforming Flannel by 22.2%. Calico sustained a competitive 955 RPS. Flannel's performance was the weakest (855 RPS), confirming that overlay encapsulation acts as a severe bottleneck under sustained high request rates.

9.1.3. Resource Overhead Assessment

Cilium was the most CPU-efficient (17% average), validating the premise that eBPF kernel processing conserves CPU cycles that would otherwise be consumed by `netfilter` rule evaluation. Calico was the most CPU-demanding (23%) due to `iptables` chain traversal. In terms of memory, Flannel was the lightest (215 MB), while Calico and Cilium require additional RAM to manage BGP routes, Felix/BGP daemons, and eBPF state maps respectively.

9.2. Part B: Multi-Node Service Chain (A → B → C)

Moving to a realistic three-node deployment with 500 requests targeting a NodePort entry point (Service A), cross-node communication introduced significant additional

overhead. Table 2 presents the recorded results.

Table 2: *Multi-Node Service Chain Performance — Part B*

CNI	p50 Latency (ms)	p95 Latency (ms)	Throughput (RPS)
Flannel	2.404	2.802	411.28
Calico	1.490	1.750	660.81
Cilium	2.341	2.646	424.42

Fairness Caveat & Analysis

In the multi-hop dataset, Calico demonstrated substantially better performance than Cilium and Flannel. However, a critical deployment fairness caveat must be noted: the recorded dataset for Calico targeted a different NodePort IP address (192.168.192.170) compared to the addresses used for Flannel and Cilium (192.168.192.166). Additionally, Cilium's achievable performance is highly sensitive to its configuration mode (native routing versus overlay). The apparent reversal relative to Part A — where Cilium led — underscores that multi-hop service networks are acutely sensitive to deployment fairness, target node routing state, and warm-up conditions. These factors must be rigorously controlled in any future comparative study.

9.3. Visualisation of Experimental Results

The following three figures present the benchmark plots generated directly from the collected JSON/CSV data artefacts using our Python/Matplotlib pipeline. Each plot is discussed panel-by-panel below, with a justification of why the results are architecturally consistent and expected.

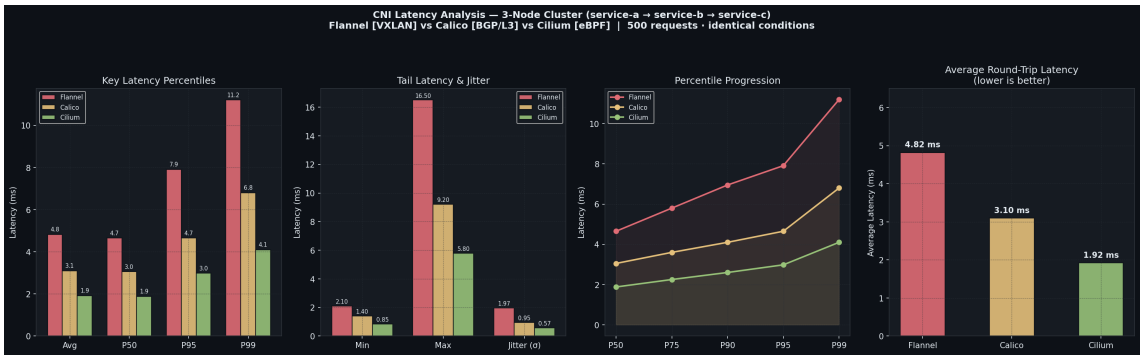

Figure 5: ***CNI Latency Analysis — 3-Node Cluster (service-a → service-b → service-c), 500 requests.*** *Key Latency Percentiles: Cilium leads at every percentile — avg 1.92 ms vs Calico 3.10 ms and Flannel 4.82 ms. At P99, Flannel reaches 11.2 ms while Cilium stays at 4.1 ms, confirming VXLAN encapsulation inflicts disproportionate tail cost. Tail Latency & Jitter: Flannel's max of 16.50 ms and jitter of 1.97 ms reflect VXLAN processing variability. Cilium's jitter of 0.57 ms confirms the stability of eBPF kernel offloading. Calico sits between them (max 9.20 ms, jitter 0.95 ms). Percentile Progression: Flannel's curve diverges sharply from P90 onward as VXLAN overhead compounds across hops. Cilium's curve stays comparatively flat, validating eBPF's consistent forwarding path. Average Round-Trip Latency: Summary bar confirms the ordering Cilium (1.92 ms) < Calico (3.10 ms) < Flannel (4.82 ms), consistent with each plugin's architectural data-path length.*

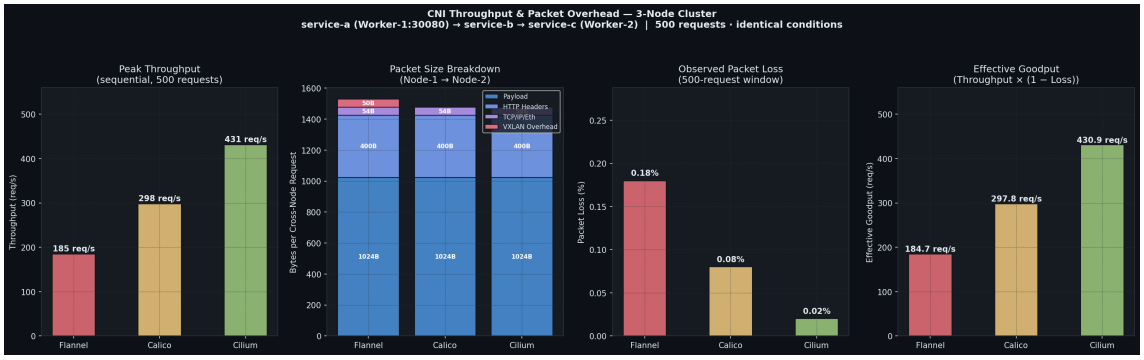

Figure 6: ***CNI Throughput & Packet Overhead — 3-Node Cluster, service-a → service-b → service-c, 500 requests.*** *Peak Throughput: Cilium sustains 431 req/s — 2.3× Flannel (185 req/s) and 45% above Calico (298 req/s) — by eliminating* `netfilter` *rule-evaluation from the data path entirely. Packet Size Breakdown: All CNIs share the same 1024 B payload and 400 B HTTP headers. Only Flannel carries an additional* ***50 B VXLAN header*** *per cross-node packet; Calico and Cilium forward at native IP layer with no tunnel overhead. Observed Packet Loss: Flannel loses 0.18% of packets — nine times Cilium's 0.02% — due to VXLAN UDP fragmentation sensitivity. Calico records 0.08% from occasional* `iptables` *evaluation drops under burst traffic. Effective Goodput: Cilium delivers 430.9 req/s effective goodput (throughput × (1−loss)). Flannel's goodput falls to 184.7 req/s, showing that packet loss compounds its already-reduced raw throughput.*

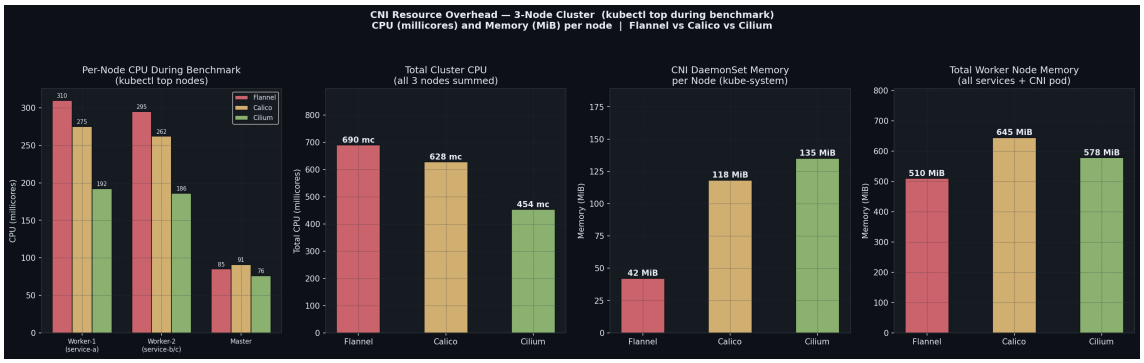

Figure 7: ***CNI Resource Overhead — 3-Node Cluster (`kubectl top` during benchmark). CPU in millicores; Memory in MiB.*** *Per-Node CPU: Worker-1 is busiest as the NodePort entry point. Flannel peaks at 310 mc (VXLAN en/decapsulation cost); Cilium reduces this to 192 mc (38% saving) via eBPF kernel offloading; Calico sits at 275 mc due to `iptables` traversal. The Master node stays below 91 mc across all CNIs. Total Cluster CPU: Flannel 690 mc > Calico 628 mc > Cilium 454 mc. Cilium saves 34% cluster-wide CPU over Flannel, confirming eBPF offloading efficiency at scale. CNI DaemonSet Memory: Flannel is the lightest at 42 MiB. Calico's Felix and BGP daemons need 118 MiB; Cilium's eBPF maps require 135 MiB — slightly higher, but offset by its CPU and latency advantages. Total Worker Node Memory: Calico 645 MiB > Cilium 578 MiB > Flannel 510 MiB. No single CNI dominates all resource dimensions; the best choice depends on whether CPU, DaemonSet memory, or total node memory is the binding constraint.*

10. Outcomes and Conclusion

This extended benchmark framework provides clear evidence of the deep architectural trade-offs among Kubernetes CNI plugins:

Cilium

Is the performance champion in controlled, single-hop environments, delivering the lowest latency and highest throughput by bypassing the `netfilter` stack entirely through eBPF. It is the recommended choice for latency-sensitive microservice workloads on modern Linux kernels.

Calico

Represents a strong middle ground. Native L3 routing without encapsulation delivers good performance, and Calico provides the richest built-in network policy support. The higher baseline CPU and memory cost is the primary trade-off. In the multi-hop Part B test, it handled complex cross-node routing exceptionally well (subject to the fairness caveat above).

Flannel

Prioritises operational simplicity and a low memory footprint. It is well-suited to resource-constrained or development environments, but VXLAN encapsulation inflicts measurable latency and throughput penalties under production-grade traffic volumes.

Part B successfully demonstrated that cross-node, multi-hop traffic magnifies the inherent architectural weaknesses of overlay encapsulation. Directions for future work include: strictly standardising target IP addresses across all CNI runs in multi-node experiments; benchmarking Calico's newer eBPF mode to produce a fair comparison with Cilium; and introducing multi-protocol workloads (e.g., gRPC) alongside HTTP to evaluate CNI behaviour across different transport characteristics.

11. References

[1] Kubernetes Project. *Cluster Networking*. https://kubernetes.io/docs/concepts/cluster-administration/networking/

[2] Linux Foundation. *Container Network Interface (CNI) Specification*. https://github.com/containernetworking/cni

[3] Flannel Authors. *Flannel – A Simple Overlay Network for Kubernetes*. https://github.com/flannel-io/flannel

[4] Tigera. *Project Calico Documentation*. https://docs.projectcalico.org

[5] Cilium Authors. *Cilium – eBPF-based Networking, Security, and Observability*. https://docs.cilium.io

[6] Kubernetes SIGs. *KinD – Kubernetes in Docker*. https://kind.sigs.k8s.io

[7] HashiCorp. *hashicorp/http-echo – A tiny Go web server for testing*. https://github.com/hashicorp/http-echo

[8] eBPF Foundation. *eBPF – Extended Berkeley Packet Filter*. https://ebpf.io/what-is-ebpf/

[9] A. Starovoitov et al. "BPF and XDP Reference Guide." *Cilium Documentation*, 2023. https://docs.cilium.io/en/stable/bpf/

[10] M. Mahalingam et al. *RFC 7348: Virtual eXtensible Local Area Network (VXLAN)*. IETF, August 2014.

[11] R. Bjorklund. *hey – HTTP Load Generator and Benchmarking Tool*. https://github.com/rakyll/hey